

Fundação Getúlio Vargas
Estrutura de Dados

Relatório Lista 2

Alunos:

Pedro Marrochio Lacerda de Abreu - 251704025

Pedro Ricardo Alves Freitas — 251708013

Professor:

Matheus Telles Werner

13 de abril de 2026

Conteúdo

1	Estrutura de dados	2
2	Operações	2
2.1	Busca por contraparte	2
2.2	Remoção de ordens	2
3	Custo computacional	2
3.1	Inserção e cancelamento	2
3.2	execução de ordens	2
3.3	recuperação das ordens/transações	2

1 Estrutura de dados

Lista duplamente encadeada (DLL) para as ordens (em que cada objeto "nó" dessa lista armazena um objeto do tipo ordem). Essa estrutura foi escolhida principalmente pelo tamanho indefinido da lista de ordens que pode crescer muito rapidamente, já que para cada ordem de compra ou venda não compatível com as já existentes que surge, ela é armazenada no sistema, ou seja, caso a estrutura escolhida fosse array dinâmico, por exemplo, o tamanho desse array pode dobrar rapidamente e ocupar a memória de modo desenfreado, o que não acontece com as DLL's, já que o espaço solicitado na memória só surge quando uma nova ordem precisa ser armazenada. Além disso, remover ordens no início, meio ou fim do array é mais fácil com a estrutura da DLL, quando ordens de compra e venda forem compatíveis e gerarem uma transação e, assim como o array, essa estrutura contribui na iteração das ordens para buscas dentre as ordens.

Array dinâmico para o armazenamento de transações (um array de ponteiros em que cada um aponta para um objeto do tipo transação), pois elas não possuem um tamanho exato, ou seja, podem existir quantas transações forem necessárias e, além disso, diferentemente das *Orders*, as transações possuem um critério de existência mais rigoroso, ocorrendo somente quando uma ordem de compra e venda são compatíveis; ou seja, uma transação, além de armazenar duas informações em um único lugar, ocorre menos que as ordens, abrindo espaço para uma estrutura que pode crescer, mas que não escala tanto quanto as ordens.

2 Operações

2.1 Busca por contraparte

Tanto a busca por quem está vendendo quanto a busca por quem está comprando ordens são feitas de modo análogo, com um loop **while** que percorre a lista inteira e a cada iteração verifica se há alguma ordem compatível com a que foi enviada, guardando, caso encontre, na variável de ordem selecionada criada antes do **while**. Nesse sentido, dentro do loop há algumas condicionais; primeiro é checado se a ordem enviada é de compra ou venda; depois procura-se por um valor menor ou igual ao da ordem submetida e, caso encontre, verifica se há alguma ordem da lista que já foi selecionada, troca caso a ordem nova tenha um valor menor e trata o caso de empate, caso aconteça. Vale ressaltar que para a variável "**selected**", que guarda a melhor ordem compatível ao longo do loop, ela precisa ser inicializada como uma "ordem base", que serve apenas como uma verificação caso nenhuma ordem compatível seja encontrada. Por esse motivo, essa ordem possui um tipo "t" a nível de sistema, o que significa que essa ordem **sempre** é trocada caso alguma outra da lista passe na verificação de preço menor ou igual, do contrário, ao fim do **while**, há uma condição que checa se a ordem selecionada é do tipo "t" e, acontecendo isso, a ordem enviada é inserida na lista.

2.2 Remoção de ordens

Tanto no **submit()** quanto no **cancel()** a remoção é feita utilizando a função **remove()** da própria estrutura da DLL, que verifica se precisa remover no início da lista e, assim, utilizando a função **removeFront()** ou no fim, utilizando a função **removeEnd()**; caso contrário, **remove()** possui sua própria lógica para remover no meio da lista como foi feita a remoção de ordens; vale ressaltar que as 3 funções possuem uma lógica parecida de modificar os ponteiros dos nós que se relacionam com aquele que será removido para que a estrutura da lista não quebre.

3 Custo computacional

3.1 Inserção e cancelamento

Ambos **O(n)**. Na inserção isso se deve ao loop **while** dentro da função **submit** que sempre percorre a lista inteira de ordens já existentes para checar se há alguma ordem melhor a cada iteração e, no final, caso não encontre, faz o **insert** em si na lista de ordens que tem um custo fixo já que uma ordem é sempre inserida no fim.

Já no cancelamento isso se deve ao loop **while** dentro da função **cancel** que, no pior dos casos, percorre a lista inteira até encontrar a ordem que será cancelada e, fora do **while**, a função **remove** da lista de ordens também é **O(n)**, pois procura elemento a elemento da lista até encontrar o que será removido.

3.2 execução de ordens

Em geral **O(1)**, pois a inserção ocorre no fim da lista, mas no pior dos casos, quando a capacidade máxima é atingida, a lista dobra de tamanho e copia os elementos já existentes para a nova, tornando-se **O(n)**.

3.3 recuperação das ordens/transações

O(n) em ambos os casos, devido ao loop **for** principal que cria o novo array de retorno elemento a elemento.